

SQL- SUBCONSULTAS

Las subconsultas en SQL Server son consultas anidadas dentro de una consulta más grande. Se utilizan para realizar consultas complejas o para filtrar datos basados en resultados de otras consultas. El concepto principal es que una subconsulta se ejecuta primero y su resultado se utiliza en la consulta principal.

Una subconsulta es simplemente una consulta SELECT que vive *adentro* de otra consulta. La consulta exterior usa el resultado de la interior para hacer su trabajo.

Existen diferentes tipos de subconsultas:

- Subconsulta Escalar
- Subconsulta de lista
- Subconsulta correlacionada

Subconsulta escalar

Una subconsulta escalar es una subconsulta que devuelve un solo valor. Esta subconsulta se utiliza en contextos donde se espera un solo valor, una condición de filtro, una comparación, o cualquier otro lugar donde normalmente se usaría un valor individual.

Ejemplo:

```
SELECT *
FROM Productos
WHERE precio > (SELECT AVG(precio) FROM Productos)
```

en la consulta anterior se solicitan todos los productos donde su precio sea mayor al promedio de precios de todos los productos.

Subconsulta de lista

Son las subconsultas que devuelven una sola columna con múltiples filas. Estas subconsultas suelen incluirse en la cláusula WHERE para filtrar los resultados de la consulta principal. Suelen ser utilizadas con el operador IN / NOT IN

Ejemplo:

```
SELECT * FROM Proveedores
WHERE idProveedor IN (SELECT idProveedor FROM Compras)
```

En esta consulta se retornan todos los proveedores a los cuales se les haya hecha alguna compra, es decir se hace un select de la tabla proveedores donde el id se encuentre en la columna idProveedor de la tabla Compras.

Subconsulta correlacionada

Una subconsulta correlacionada es una subconsulta que hace referencia a una o más columnas de la tabla de la consulta principal. Esta referencia permite que la subconsulta sea evaluada de forma iterativa para cada fila de la tabla externa. La consulta interna se ejecuta para cada fila de la tabla externa.

Ejemplo:

```
SELECT
c.razonSocial,
(SELECT COUNT(v.idventa) from ventas v where v.idCliente = c.idCliente) as Cantidad
FROM Clientes C
```

La subconsulta se coloca en la cláusula SELECT porque queremos obtener una columna adicional con el número de ventas realizadas por el cliente correspondiente. Para cada registro de la tabla Clientes la subconsulta interna calcula la cantidad de ventas realizadas por un cliente con el ID correspondiente.

Las subconsultas correlacionadas pueden ser también utilizadas en la cláusula WHERE, por ejemplo:

```
SELECT razonSocial FROM Proveedores p
WHERE EXISTS(SELECT * FROM Compras C WHERE C.idProveedor = p.idProveedor)
```

Esta consulta obtiene los proveedores a los cuales no se les haya comprado nunca y funciona de la siguiente manera:

1. La consulta externa la razón social de los proveedores.
2. La consulta interna busca registros que se correspondan con el ID del proveedor que está siendo comprobado por la consulta externa, mediante el operador EXISTS.
3. Si hay registros correspondientes, se añade a la salida la razón social del proveedor correspondiente.

Las subconsultas, generalmente, pueden ser reemplazadas por combinaciones (JOINS). De ser posible, se aconseja emplear JOINS en lugar de subconsultas ya que suelen ser más eficientes, aunque no siempre puede reemplazarse una subconsulta con un join. Por el contrario, los joins siempre pueden ser reemplazados por una subconsulta.

Operadores clave con subconsultas

IN / NOT IN

IN

Es un operador que se usa para **comparar un valor contra un conjunto de valores** (generalmente devueltos por una subconsulta o lista). Verifica si un valor esta dentro de una lista. Internamente funcionan como múltiples OR

Ejemplo:

```
SELECT *
FROM Clientes
WHERE IdCliente IN (1, 3, 5)
```

Es lo mismo que:

```
SELECT *
FROM Clientes
WHERE IdCliente = 1
OR IdCliente = 3
OR IdCliente = 5
```

NOT IN

Es la negación de IN, es decir que devuelve los resultados cuyos valores no se encuentren dentro de la lista. Internamente funciona como múltiples AND.

Ejemplo:

```
SELECT *
FROM Clientes
WHERE IdCliente NOT IN (1, 3, 5)
```

Es lo mismo que:

```
SELECT *
FROM Clientes
WHERE IdCliente <> 1
AND IdCliente <> 3
AND IdCliente <> 5
```

Se debe tener especial cuidado cuando se utiliza NOT IN como operador. Con valores concretos es directo:

```
SELECT Nombre FROM Empleados WHERE IdDepartamento NOT IN (1, 2, 3);
```

Con subconsulta funciona igual... hasta que aparece un NULL

Cuando la lista viene de una subconsulta, NOT IN puede no retornar lo que se desea:

```
SELECT Nombre
FROM Empleados
WHERE IdDepartamento NOT IN ( SELECT IdDepartamento FROM Departamentos );
```

Si la subconsulta devuelve aunque sea un solo NULL, el resultado completo es 0 filas. Sin error, sin aviso.

¿Por qué pasa esto?

La clave está en cómo SQL evalúa las comparaciones con NULL.

En SQL, NULL no es un valor: es la ausencia de valor. Por eso ninguna comparación con NULL devuelve TRUE o FALSE, sino UNKNOWN:

Internamente SQL expande la condición así:

```
WHERE IdDepartamento <> 1
AND IdDepartamento <> 2
AND IdDepartamento <> NULL ← siempre UNKNOWN
```

Esa última condición nunca puede ser TRUE. Y como todas las condiciones están unidas con AND, toda la expresión queda en UNKNOWN. Las filas UNKNOWN no pasan el filtro, así que **no devuelve ninguna fila**.

Como usar NOT IN con subconsulta de forma segura:

Si podés garantizar que la subconsulta no va a devolver NULLs, NOT IN funciona perfectamente y es más legible. La mejor forma de garantizarlo es agregar WHERE column IS NOT NULL en la subconsulta:

```
SELECT Nombre
FROM Empleados
WHERE IdDepartamento NOT IN ( SELECT IdDepartamento FROM Departamentos WHERE IdDepartamento IS NOT NULL -- ← blindaje
);
```

EXISTS / NOT EXISTS

EXISTS

Se usan con subconsultas para verificar si existen o no registros relacionados.

A diferencia de IN, **no comparan valores**, sino que verifican **existencia de filas**.

Devuelve TRUE si la subconsulta devuelve **al menos una fila**.

```
SELECT *
FROM Clientes c
WHERE EXISTS (
  SELECT 1
  FROM Ventas v
  WHERE v.IdCliente = c.IdCliente
)
```

Trae clientes que **tienen al menos una venta**

¿Cómo se ejecuta internamente?

Para cada cliente:

- SQL evalúa la subconsulta
- Si encuentra una fila, se detiene
- Ya considera el resultado como TRUE
- No necesita traer todos los datos
- Solo le importa si existe algo

Interpretación mental: EXISTS = “¿existe al menos uno?”

NOT EXISTS

Devuelve TRUE si la subconsulta **NO devuelve ninguna fila**

```
SELECT *
FROM Clientes c
WHERE NOT EXISTS (
  SELECT 1
  FROM Ventas v
  WHERE v.IdCliente = c.IdCliente
)
```

Para cada cliente:

Si encuentra una fila → lo descarta

Si no encuentra ninguna → lo incluye

Interpretación mental: NOT EXISTS = “no existe ninguno”

Tanto en EXISTS como en NOT EXISTS se suele usar SELECT 1 por convención, ya que no importa lo que devuelva sino que devuelva algo.

ANY / SOME / ALL

ANY / SOME

Son operadores que se usan con subconsultas para **comparar un valor contra un conjunto de valores**.

SOME es sinónimo de ANY (hacen exactamente lo mismo), La condición se cumple si es verdadera para **al menos UN valor**.

```
SELECT *  
FROM Productos  
WHERE Precio > ANY (SELECT Precio FROM Productos WHERE Categoria = 'A')
```

Supongamos que la subconsulta devuelve: 10,20,30

Entonces SQL evalúa:

```
Precio > 10  
OR Precio > 20  
OR Precio > 30
```

> ANY = **mayor que alguno**

ALL

La condición se cumple si es verdadera para **TODOS los valores**

```
SELECT *  
FROM Productos  
WHERE Precio > ALL (SELECT Precio FROM Productos WHERE Categoria = 'A')
```

Si la subconsulta devuelve: 10, 20, 30

Entonces:

```
Precio > 10  
AND Precio > 20  
AND Precio > 30
```

Tiene que cumplir **todas**

> ALL = **mayor que todos**